

ResourceOps - GitOps 기반 클라우드 운영 및 리소스 최적화 플랫폼

AWS EKS 기반 GitOps 환경을 구축하고, Prometheus·Grafana·CloudWatch를 활용하여 애플리케이션 및 인프라 리소스를 모니터링하며 비용 최적화 방안을 제공하는 클라우드 운영 플랫폼 프로젝트

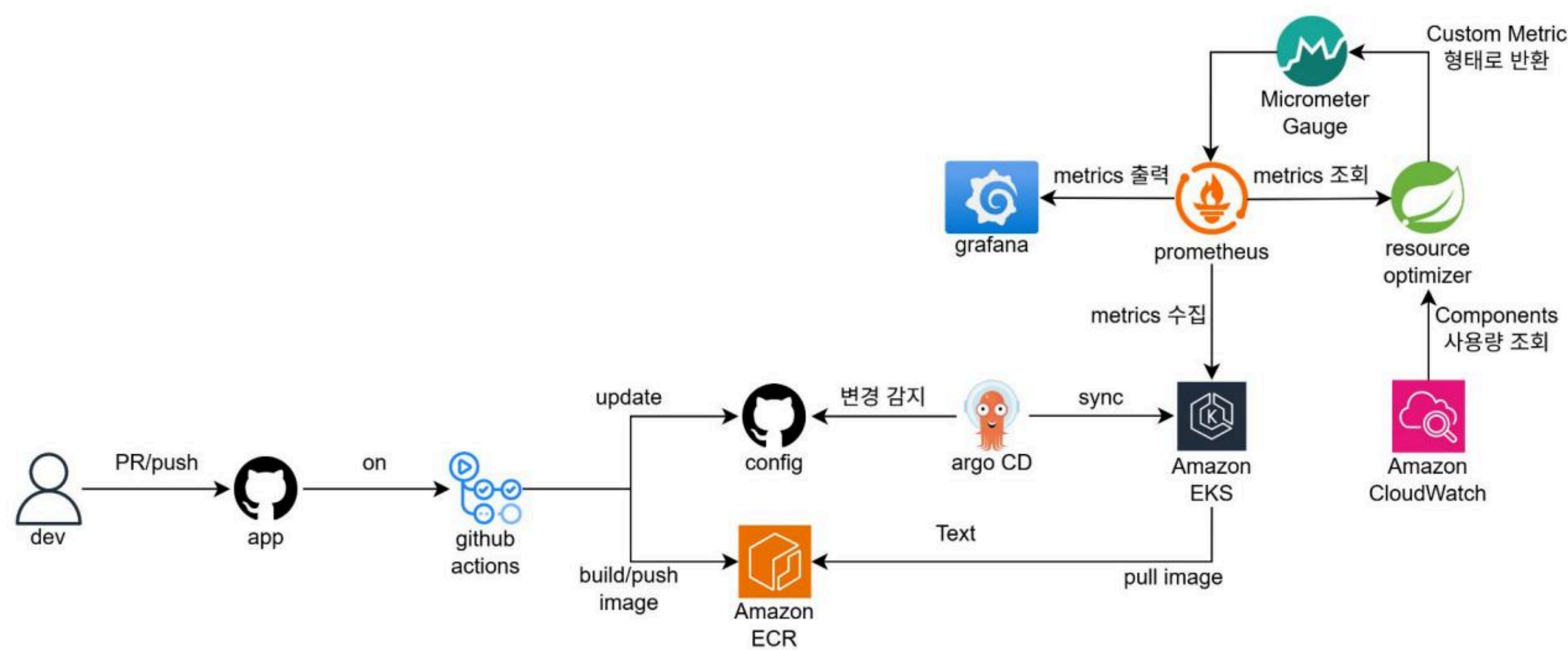
기술 스택

IaC - Terraform - AWS Provider - S3 Backend - DynamoDB Lock Cloud Infrastructure - AWS VPC - AWS EKS - AWS RDS (MySQL) - AWS ECR - AWS IAM - AWS SSM Parameter Store - AWS CloudWatch Container & Orchestration - Docker - Kubernetes - Amazon EKS - Gateway API - AWS Load Balancer Controller GitOps & CI/CD - GitHub Actions - ArgoCD - ApplicationSet - Kustomize Monitoring & Observability - Prometheus - Grafana - kube-prometheus-stack - CloudWatch Metrics	Security & Secret Management - IAM Role - IRSA (IAM Roles for Service Accounts) - External Secrets Operator - AWS Systems Manager Parameter Store Backend - Java 17 - Spring Boot Collaboration - Git - GitHub - Notion
--	---

프로젝트 개요

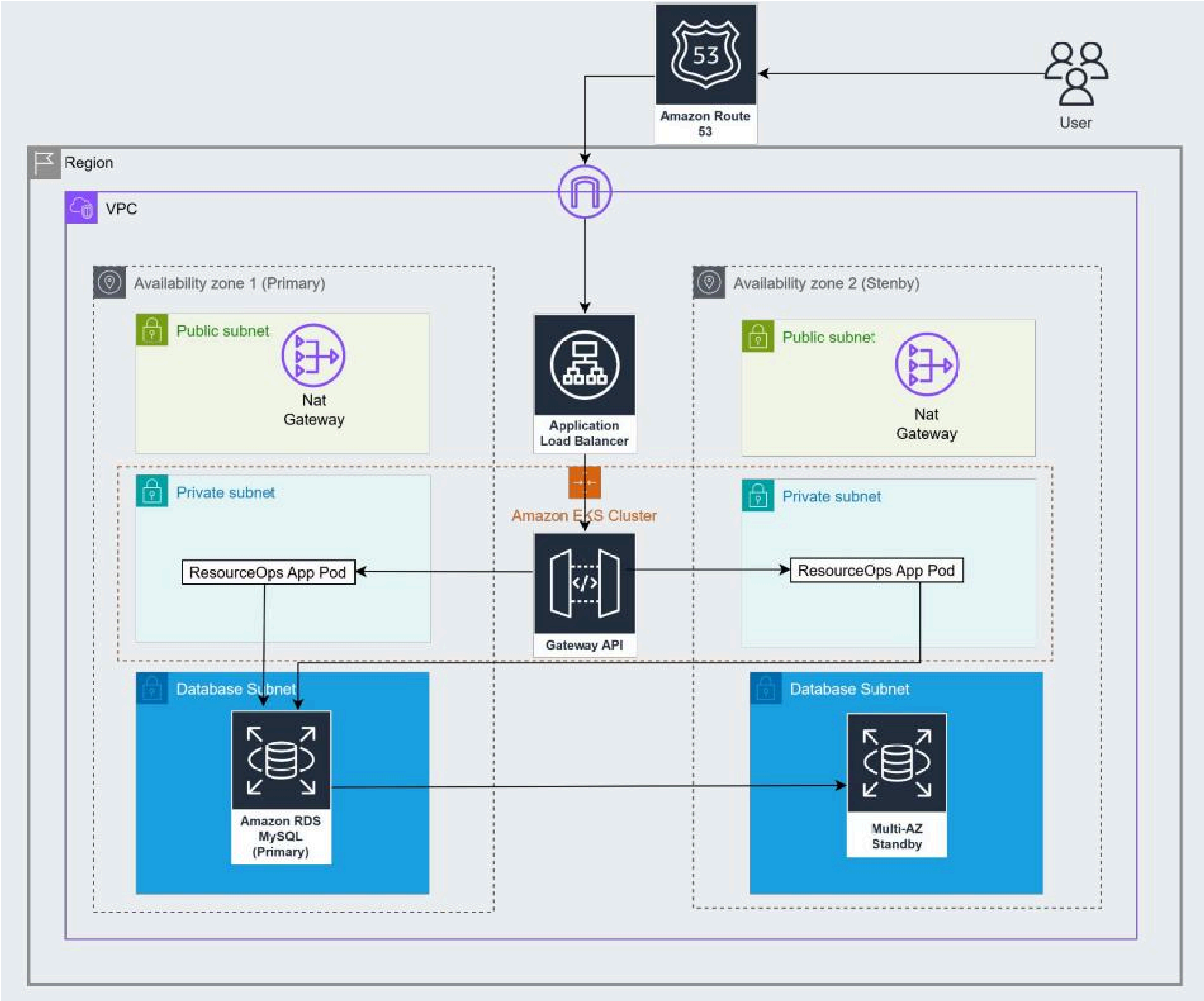
- 프로젝트명
ResourceOps - GitOps 기반 클라우드 운영 및 리소스 최적화 플랫폼
- 프로젝트 소개
AWS EKS 기반 GitOps 환경을 구축하고 애플리케이션 및 인프라 리소스를 모니터링하여 운영 효율과 비용 최적화를 지원하는 클라우드 운영 플랫폼 프로젝트입니다. Terraform 활용한 AWS 인프라 구축, GitHub Actions·ArgoCD 기반 CI/CD 파이프라인 구현, Prometheus·Grafana·CloudWatch를 활용한 모니터링 환경 구축을 수행하였으며, 수집된 메트릭을 기반으로 리소스 사용 패턴 분석 및 비용 최적화 시뮬레이션 기능을 구현하였습니다.
- 개발 인원
6명
- 담당 역할
 - Team Leader
 - i. 프로젝트 일정 관리 및 팀원 역할 분담
 - ii. 전체 시스템 아키텍처 설계 주도
 - iii. GitOps 기반 개발 프로세스 수립
 - Monitoring & Optimization
 - i. Prometheus·Grafana 기반 모니터링 환경 구축
 - ii. CloudWatch 메트릭 수집 및 비용 데이터 분석
 - iii. 리소스 최적화 및 비용 절감 시뮬레이션 구현
 - Kubernetes
 - i. EKS 애플리케이션 배포 및 운영 환경 구성
 - ii. Gateway API 기반 트래픽 라우팅 구성
 - iii. Kubernetes 리소스 운영 및 모니터링

기능흐름



- **애플리케이션 코드 변경:**
개발자가 App Repository에 코드를 Push 또는 PR 생성
- **CI 파이프라인 실행:**
GitHub Actions가 자동 실행되어 애플리케이션 빌드 및 테스트 수행
Docker 이미지를 생성한 후 Amazon ECR에 Push
- **GitOps 배포 트리거:**
GitHub Actions가 Config Repository의 이미지 태그를 최신 버전으로 업데이트
ArgoCD가 Config Repository의 변경 사항을 감지
- **애플리케이션 배포:**
ArgoCD가 변경된 Kubernetes Manifest를 Amazon EKS에 동기화(Sync)
EKS는 Amazon ECR에서 최신 이미지를 Pull하여 애플리케이션을 배포
- **모니터링 및 메트릭 수집**
Prometheus가 EKS 클러스터와 애플리케이션의 CPU, Memory, Pod 상태 등의 메트릭을 수집
Grafana를 통해 실시간 운영 현황을 시각화
- **리소스 사용량 분석:**
Resource Optimizer가 Prometheus 메트릭과 CloudWatch 비용 데이터를 조회
- **비용 최적화 결과 제공:**
결과를 Custom Metric 형태로 Prometheus에 전달
Grafana 대시보드에서 리소스 최적화 플랜 적용값 시뮬레이션과 동시에 관찰하며 비용 절감 효과 확인

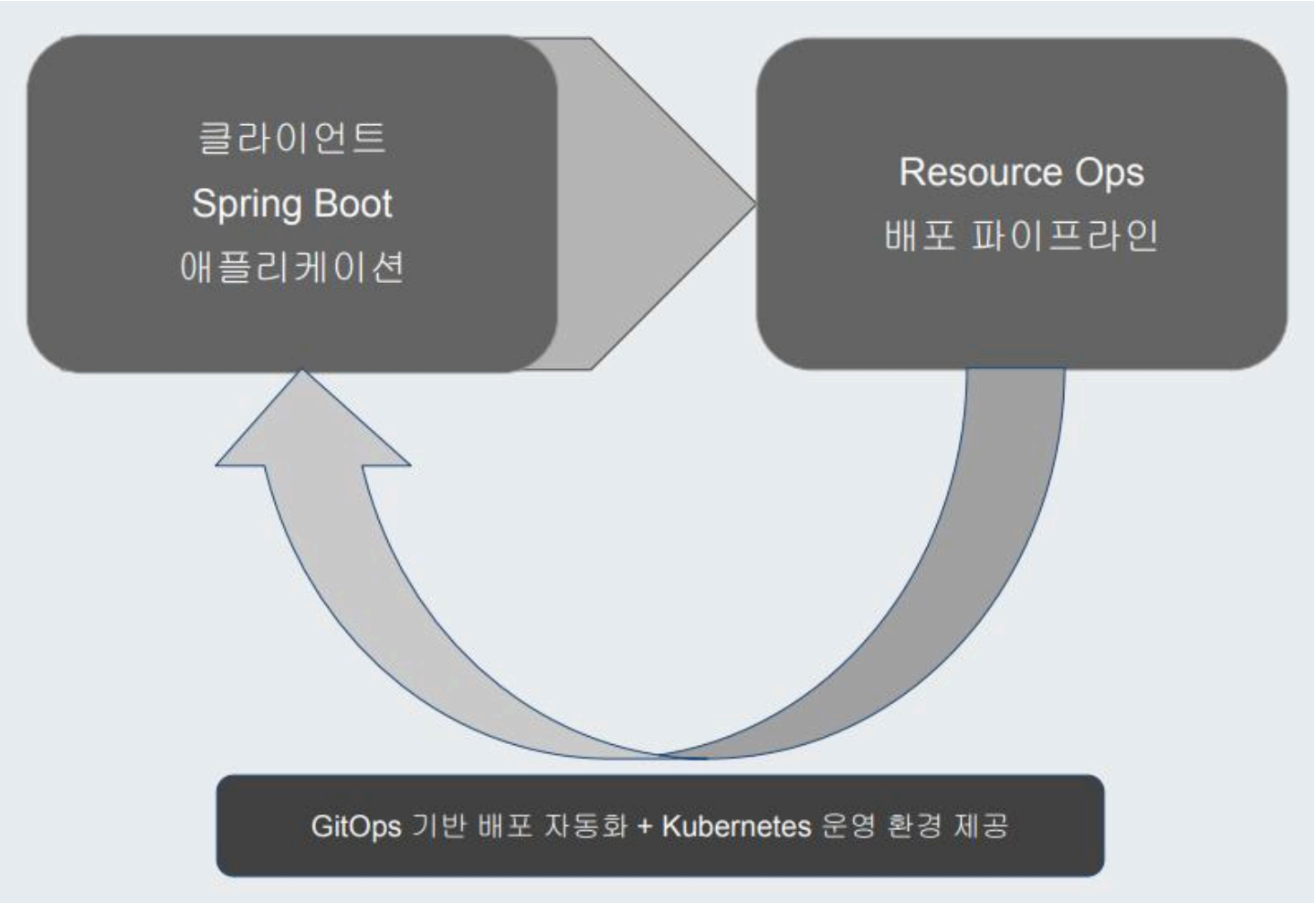
AWS 아키텍처



구조 특징

- Terraform 기반 IaC를 활용한 AWS 인프라 자동화
- Multi-AZ 환경을 통한 EKS 및 RDS 고가용성 구성
- ALB + Gateway API 기반 외부 트래픽 라우팅
- Private Subnet 기반 애플리케이션 운영
- Amazon ECR 연동 컨테이너 이미지 관리
- CloudWatch·Prometheus·Grafana 기반 통합 모니터링
- GitOps 기반 자동 배포 및 운영 환경 구축

주요기능

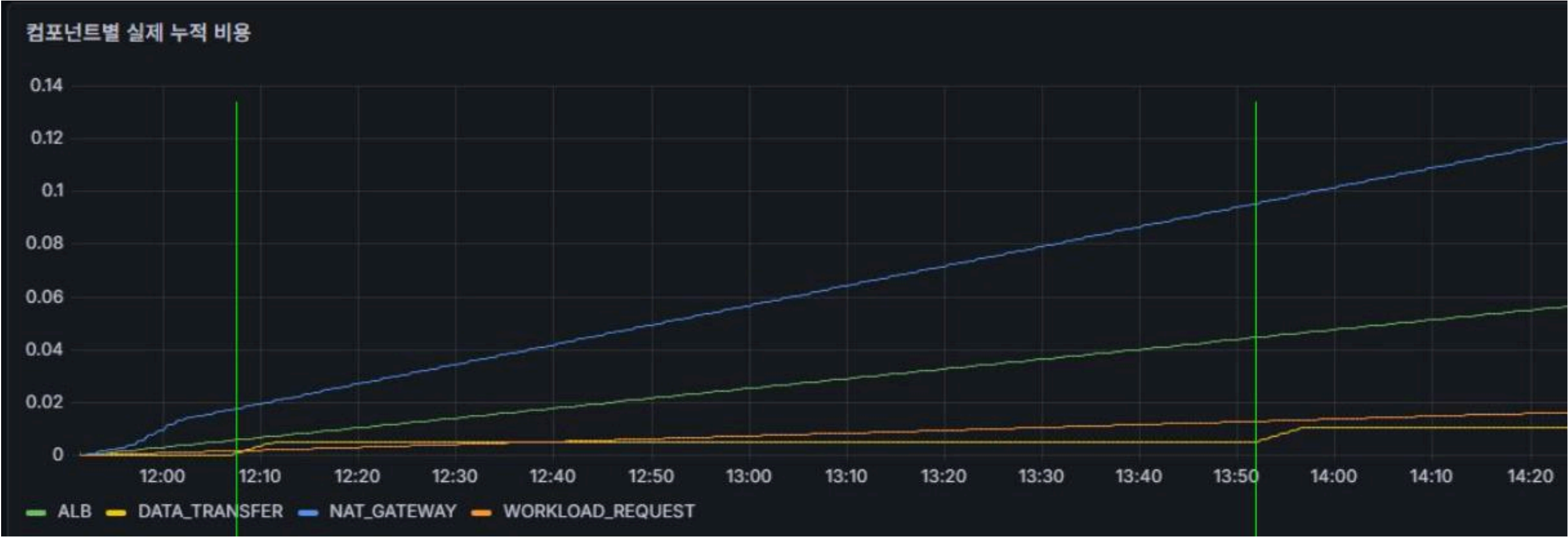


클라이언트 서비스 통합 배포

- 사용자의 애플리케이션을 ResourceOps 플랫폼에 통합하여 GitOps 기반 자동 배포 환경과 Kubernetes 운영 환경을 제공. 이를 통해 개발자는 인프라 구성 부담 없이 애플리케이션 배포 및 운영을 수행

서비스 리소스 비용 모니터링

- CloudWatch 비용 데이터를 기반으로 ALB, NAT Gateway, Data Transfer 등 주요 리소스의 비용 발생 현황을 시각화

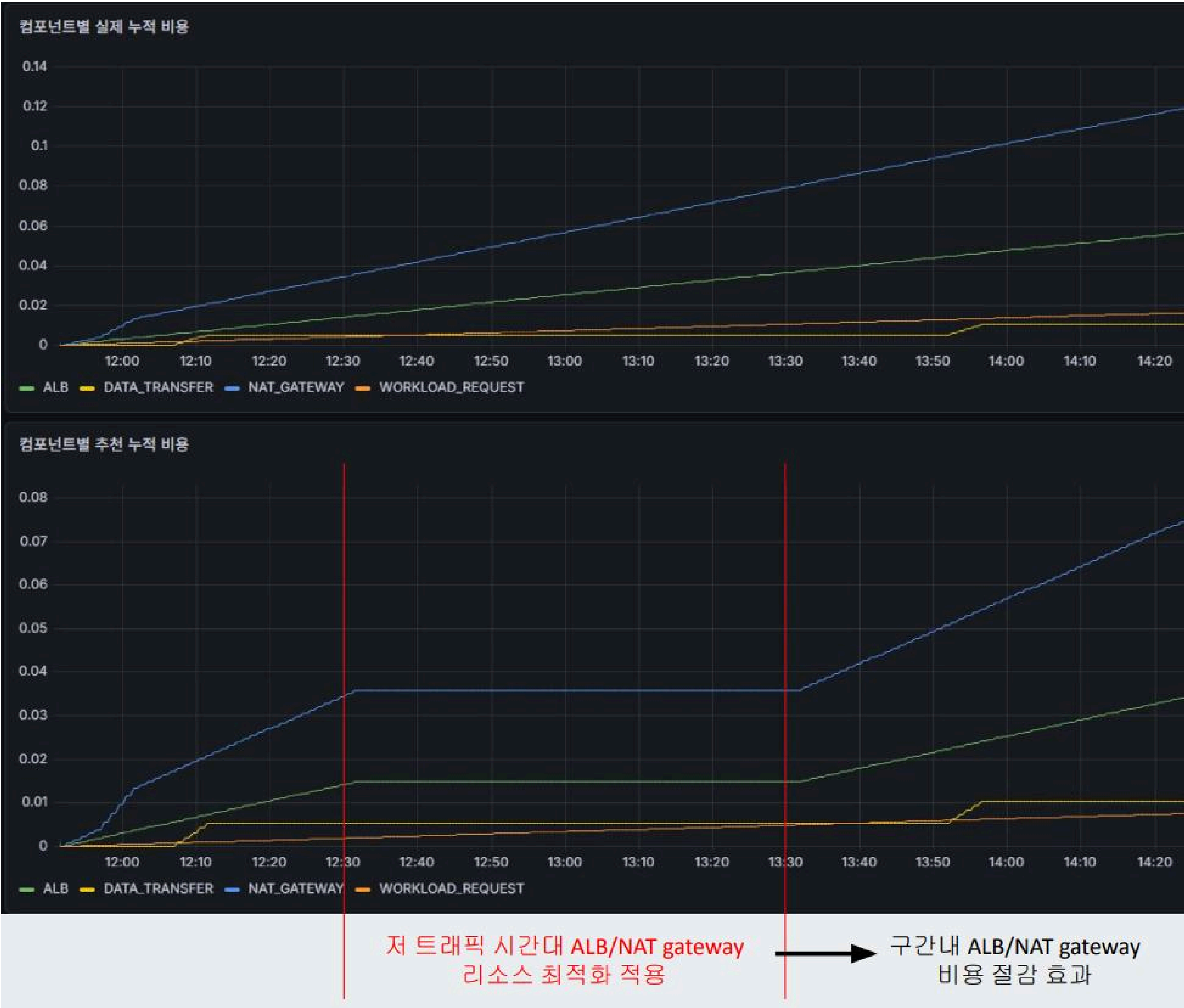


두 번의 Load Test 진행 후 Data Transfer 비용이 부과 확인

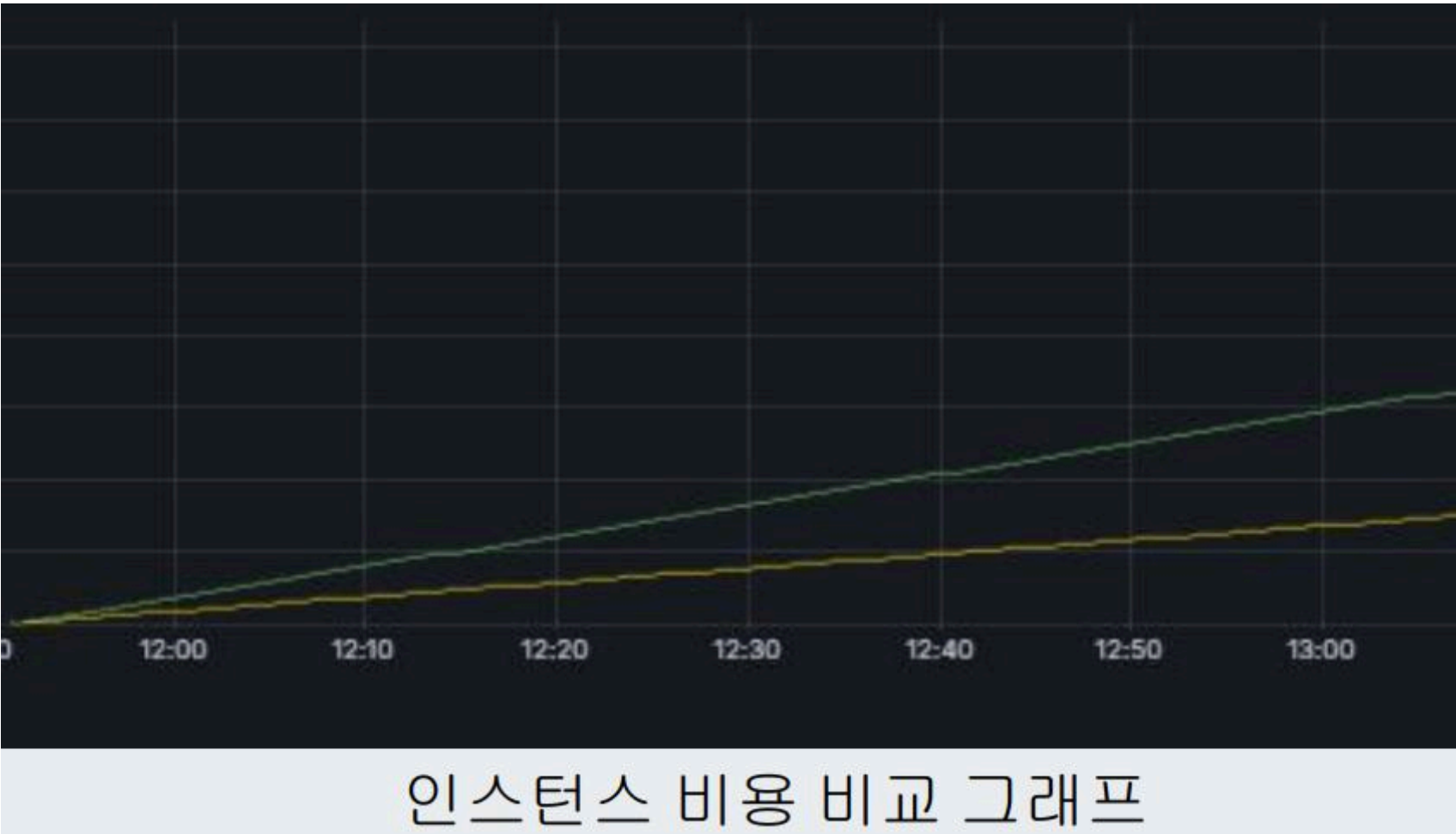
주요기능

리소스 최적화 시뮬레이션 비교

- 리소스 비용 최적화 정책 적용 시뮬레이션과 실제 비용을 비교하면서 비용 절감 확인
- 시뮬레이션에 특정 저트래픽 시간대에 ALB/NAT gateway 비활성화 적용 후 해당 시간대 ALB/NAT gateway에 비용이 부과되지 않는 효과 확인



- CPU/Memory request를 조정해 인스턴스 절감 효과 또한 확인가능



추후 개선사항

- **비용 계산 로직 정교화**
 - 서비스별 비용 산정 정확도 향상을 위해 비용 계산 로직 고도화
 - 리소스별 비용 기여도 분석 기능 추가
- **트래픽 기반 비운영 시간대 자동 설정**
 - 트래픽 패턴 분석을 통한 비운영 시간대 자동 산출 기능 구현
 - ALB 및 NAT Gateway 운영 시간 자동 최적화 기능 추가
- **추천 알고리즘 개선**
 - 장기간 운영 데이터 기반 리소스 추천 알고리즘 고도화
 - 트래픽 변동성을 고려한 비용 최적화 정확도 향상
- **4. HPA 및 Replica 최적화 추천**
 - CPU·Memory 사용량 기반 적정 HPA 설정값 추천 기능 추가
 - 서비스별 최적 Replica 수 자동 산출 기능 구현
- **5. AWS 비용 컴포넌트 확장**
 - EBS, RDS, CloudWatch, ECR 등 추가 AWS 비용 항목 분석 지원 예정
 - 서비스 단위 비용 분석 범위 확대 예정